

oreilly.notes.dutc.io

'Integrate LLMs with Python'

A Seminar by 'Don't Use This Code'

DO(N'T) USE THIS C<>DE

Presenter: James Powell james@dutc.io

Join us on [Discord](https://discord.gg/ZhJPKYSfNp)
([https://discord.gg/
ZhJPKYSfNp](https://discord.gg/ZhJPKYSfNp)) for
discussion and guidance!

Contents

1. [A Seminar by 'Don't Use This Code'](#)
2. [Book a Class!](#)
3. [About](#)
 - [Don't Use This Code; Training & Consulting](#)

Book a Class!

Book a class or training for your team!

Please reach out to us at learning@dutc.io if are interested in bringing this material, or any of our other material, to your team.

We have courses on topics such as:

- intro Python
- expert Python
- data engineering with Python
- data science and scientific computing with `numpy` , `pandas` , and `xarray`

If you reach out to us, we can also provide a printable copy of the notes, cleaned-up and in .pdf format, as well as a professionally edited video recording of this presentation.

About

Don't Use This Code; Training & Consulting

Don't Use This Code is a professional training, coaching, and consulting company. We are deeply invested in the open source scientific computing community, and are dedicated to bringing better processes, better tools, and better understanding to the world.

Don't Use This Code is growing! We are currently seeking new partners, new clients, and new engagements for our expert consulting and training services.

Our ideal client is an organization, large or small, using open source technologies, centering around the PyData stack for scientific and numeric computing. Organizations looking to better employ these tools would benefit from our wide range of training courses on offer, ranging from an intensive introduction to Python fundamentals to advanced applications of Python for building large-scale, production systems. Working with your team, we can craft targeted curricula to meet your training goals. We are also available for consulting services such as building scientific computing and numerical analysis systems using technologies like Python and React.

We pride ourselves on delivering top-notch training. We are committed to providing quality training that is uniquely valuable to each individual attendee, and we do so by investing in three key areas: our content, our processes, and our contributors.

Notes

```
python -m pip install httpx fastapi aiosqlite aiofiles aiopg psycpg2 pandas numpy scipy n
python -m pip install httpie
```

Starting Simply

Key Questions

- How do we use `httpie` (or `curl`) to interact with the OpenAI REST API?
- How do we use `python` (with `httpx`) to interact with the OpenAI REST API?
- What simple automation might we need to create to support this use-case?

```
print("Let's take a look !")
```

```
readonly root_url='https://api.openai.com/v1'
typeset -Ar headers=(
    Authorization "Bearer ${OPENAI_API_KEY}"
)
typeset -a httpie_args=(
    "${(@@A)}${(for k v ( "${(@kv)headers}" ) printf '%s:%s\\0' "${k}" "${v}")"%$'\0'}"
)
http get "${root_url}"/models "${(@)httpie_args}" | jq '.data | .[] | .id' | sort
```

```
"babbage-002"
"chatgpt-4o-latest"
"dall-e-2"
"dall-e-3"
"davinci-002"
"gpt-3.5-turbo"
"gpt-3.5-turbo-0125"
"gpt-3.5-turbo-1106"
"gpt-3.5-turbo-16k"
"gpt-3.5-turbo-instruct"
"gpt-3.5-turbo-instruct-0914"
"gpt-4"
"gpt-4-0125-preview"
"gpt-4-0613"
"gpt-4-1106-preview"
"gpt-4.5-preview"
"gpt-4.5-preview-2025-02-27"
"gpt-4o"
"gpt-4o-2024-05-13"
"gpt-4o-2024-08-06"
```

"gpt-4o-2024-11-20"
"gpt-4o-audio-preview"
"gpt-4o-audio-preview-2024-10-01"
"gpt-4o-audio-preview-2024-12-17"
"gpt-4o-mini"
"gpt-4o-mini-2024-07-18"
"gpt-4o-mini-audio-preview"
"gpt-4o-mini-audio-preview-2024-12-17"
"gpt-4o-mini-realtime-preview"
"gpt-4o-mini-realtime-preview-2024-12-17"
"gpt-4o-mini-search-preview"
"gpt-4o-mini-search-preview-2025-03-11"
"gpt-4o-mini-transcribe"
"gpt-4o-mini-tts"
"gpt-4o-realtime-preview"
"gpt-4o-realtime-preview-2024-10-01"
"gpt-4o-realtime-preview-2024-12-17"
"gpt-4o-search-preview"
"gpt-4o-search-preview-2025-03-11"
"gpt-4o-transcribe"
"gpt-4-turbo"
"gpt-4-turbo-2024-04-09"
"gpt-4-turbo-preview"
"o1"
"o1-2024-12-17"
"o1-mini"
"o1-mini-2024-09-12"
"o1-preview"
"o1-preview-2024-09-12"
"o1-pro"
"o1-pro-2025-03-19"
"o3-mini"
"o3-mini-2025-01-31"
"omni-moderation-2024-09-26"
"omni-moderation-latest"
"text-embedding-3-large"
"text-embedding-3-small"
"text-embedding-ada-002"
"tts-1"
"tts-1-1106"
"tts-1-hd"
"tts-1-hd-1106"
"whisper-1"

```
readonly root_url='https://api.openai.com/v1'  
typeset -Ar headers=(  
    Authorization "Bearer ${OPENAI_API_KEY}"
```

```

)
typeset -Ar payload=(
    model gpt-4o
    input 'What is 1+1?'
)
typeset -a httpie_args=(
    "${(@OA)}$(for k v ( "${(@kv)}headers}" ) printf '%s:%s\0' "${k}" "${v}")"%$\0'"
    "${(@OA)}$(for k v ( "${(@kv)}payload}" ) printf '%s=%s\0' "${k}" "${v}")"%$\0'"
)
http post "${root_url}"/responses "${(@)httpie_args}"

```

```

from asyncio import run
from httpx import AsyncClient
from os import environ

@lambda main: run(main())
async def main(
    root_url='https://api.openai.com/v1',
    openai_api_key=environ['OPENAI_API_KEY'],
):
    headers = {
        'Authorization': f'Bearer {openai_api_key}',
        'Content-Type': 'application/json',
    }
    json = {
        'model': 'gpt-4o',
        'input': 'What is 1+1, 1 - 1, 1 * 1 , 1 / 1, 1 % 1, 1 ** 1?',
        'instructions': 'Answer in Chinese. Use Chinese numerals. Do not add any other pun
    }
    async with AsyncClient() as client:
        resp = await client.post(f'{root_url}/responses', headers=headers, json=json)
        from pprint import pprint
        pprint(resp.json()['output'])

```

```

from asyncio import run
from httpx import AsyncClient
from os import environ

@lambda main: run(main())
async def main(
    root_url='https://api.openai.com/v1',
    openai_api_key=environ['OPENAI_API_KEY'],
):
    headers = {
        'Authorization': f'Bearer {openai_api_key}',

```

```

        'Content-Type': 'application/json',
    }
    json = {
        'model': 'gpt-4o',
        'input': '''
Who is the highest paid employee(s)? Provide multiple if there are more than one.
Alice    - 100,000
Bob      - 100,000
Charlie  - 50,000
''',
        'instructions': '''
        Provide just the answer, no narrative.
        Don't provide any additional context.
        ''',
    }
}
async with AsyncClient() as client:
    resp = await client.post(f'{root_url}/responses', headers=headers, json=json)
    from pprint import pprint
    pprint(resp.json()['output'])

```

```

from asyncio import run
from httpx import AsyncClient
from os import environ

```

```

@lambda main: run(main())
async def main(
    root_url='https://api.openai.com/v1',
    openai_api_key=environ['OPENAI_API_KEY'],
):
    headers = {
        'Authorization': f'Bearer {openai_api_key}',
        'Content-Type': 'application/json',
    }
    json = {
        'model': 'gpt-4o',
        'input': '''
        Who is the highest paid employee(s) who is not a manager?
        ''',
        'instructions': '''
        Answer questions by writing the SQL that would answer it.
        Use the following SQL table layout to answer the question.
        Don't format as Markdown. No, really, please don't use Markdown.
        Please don't write SQL in all caps (It's not the 1970s.)

        create table if not exists employees (
            name text
            , salary real

```

```

        );

        create table if not exist roles (
            name text
            , role text
        );
    '''
}

async with AsyncClient() as client:
    resp = await client.post(f'{{root_url}}/responses', headers=headers, json=json)
    from pprint import pprint
    pprint(resp.json()[ 'output' ][0][ 'content' ][0][ 'text' ])

async def main():
    async with AsyncClient() as client, async with connect(':memory:') as conn:
        while True:
            resp = await client.post(...)
            code = resp.json()[ 'output' ][0][ 'content' ][0][ 'text' ]
            print(f'{{code = }}')
            try:
                for row in conn.execute(code):
                    print(f'{{row = }}')
            except Exception as e:
                msg = repr(e)
                continue
            else:
                break

async def main():
    async with AsyncClient() as client, async with connect(':memory:') as conn:
        for model in [...]:
            for temp in [...]:
                resp = await client.post(...)
                code = resp.json()[ 'output' ][0][ 'content' ][0][ 'text' ]
                print(f'{{code = }}')
    ...

```

Building Infrastructure

Key Questions

- What simple infrastructure do we need to build on top of our LLMs to make them useful for automated work?

- How can we implement simple caching, auditing, and budgeting for LLMs agents?

```
print("Let's take a look !")
```

```
from asyncio import run
from collections import namedtuple
from contextlib import asynccontextmanager
from dataclasses import dataclass
from os import environ
from pickle import dumps, loads
from random import Random
from textwrap import dedent

from aiosqlite import connect
from httpx import AsyncClient

class Employee(namedtuple('EmployeeBase', 'name salary')):
    @classmethod
    def from_random(cls, name, *, random_state=None):
        rnd = random_state if random_state is not None else Random()
        return cls(
            name=name,
            salary=50_000 + round(rnd.uniform(0, 100_000), -3),
        )

@asynccontextmanager
async def dummy_data(db, *, random_state=None):
    rnd = random_state if random_state is not None else Random()
    employees = [
        Employee.from_random(name=name, random_state=rnd)
        for name in 'Alice Bob Charlie Dana Evan Frank Gina Harry'.split()
    ]
    create_table = dedent('''
        create table if not exists employees (
            name text
            , salary real
        );
    ''').strip()
    insert = dedent('''
        insert into employees (name, salary) values (:name, :salary)
    ''').strip()
    await db.execute(create_table)
    await db.executemany(insert, (x._asdict() for x in employees))
    yield

@dataclass
```

```

class Cache:
    db : ...

    async def get(self, key):
        request = dumps(key)
        query = '''
            select response from requests where request = :request limit 1
        '''
        async for response, in await self.db.execute(query, {'request': request}):
            return loads(response)
    async def set(self, key, value):
        request = dumps(key)
        response = dumps(value)
        query = '''
            insert into requests (request, response) values (:request, :response)
        '''
        await self.db.execute(query, {'request': request, 'response': response})
        await self.db.commit()
    async def has(self, key):
        request = dumps(key)
        query = '''
            select response from requests where request = :request limit 1
        '''
        async for response, in await self.db.execute(query, {'request': request}):
            return True
        return False

    @classmethod
    @asynccontextmanager
    async def from_sqlite3(cls, filename):
        async with connect(filename) as db:
            query = '''
                create table if not exists requests (
                    request blob
                    , response blob
                )
            '''
            await db.execute(query)
            yield cls(db=db)

    async def run_queries(
        *questions,
        cache=None,
        root_url='https://api.openai.com/v1',
        openai_api_key=envirom['OPENAI_API_KEY'],
    ):
        rnd = Random(0)
        async with connect(':memory:') as db, AsyncClient() as client:

```

```

async with dummy_data(db, random_state=rnd):
    layout = ';\n'.join([
        x async for x, in await db.execute('select sql from sqlite_schema')
    ])

    url = f'{root_url}/responses'
    headers = {
        'Authorization': f'Bearer {openai_api_key}',
        'Content-Type': 'application/json',
    }
    for q in questions:
        json = {
            'model': 'gpt-4o',
            'input': q,
            'instructions': dedent(f'''
                Answer the specified question by writing SQL.
                Assume SQLite.
                Assume the below SQL table layout.
                Answer with just the SQL code.
                Do not add Markdown fenced code block garbage.

                {layout}
            ''').strip(),
        }
        if cache is not None and (await cache.has(json)):
            resp = await cache.get(json)
        else:
            resp = (await client.post(url, headers=headers, json=json)).json()
            if cache is not None:
                await cache.set(json, resp)
        tokens = resp['usage']['total_tokens']
        answer = resp['output'][0]['content'][0]['text']
        answer = answer.replace('```sql', '').replace('```', '')
        print(
            'Question'.center(40, '\N{box drawings light horizontal}'),
            q,
            'Query'.center(40, '\N{box drawings light horizontal}'),
            answer,
            'Cost'.center(40, '\N{box drawings light horizontal}'),
            tokens,
            'Answer'.center(40, '\N{box drawings light horizontal}'),
            sep='\n',
        )
    async for row in await db.execute(answer):
        print(f'{row = }')

```

```

async def main(filename, task, *args, **kwargs):
    async with Cache.from_sqlite3(filename) as cache:

```

```

    await task(*args, cache=cache, **kwargs)

if __name__ == '__main__':
    run(main(
        '/tmp/llm-cache.sqlite3',
        run_queries,
        'Who is the highest paid employee(s)?',
        # 'Who is the lowest paid employee?',
        # 'Who is the second highest paid employee?',
        # 'Which employees have the largest gap between salaries?',
    ))

```

Building Systems Around Our LLMs

Key Questions

- How do we effectively integrate LLM API calls into larger systems?

```

print("Let's take a look !")

```

```

from contextlib import asynccontextmanager
from collections import namedtuple
from os import environ
from random import Random
from textwrap import dedent

from aiosqlite import connect
from httpx import AsyncClient
from fastapi import FastAPI
from pydantic import BaseModel
from uvicorn import run

class Employee(namedtuple('EmployeeBase', 'name salary')):
    @classmethod
    def from_random(cls, name, *, random_state=None):
        rnd = random_state if random_state is not None else Random()
        return cls(
            name=name,
            salary=50_000 + round(rnd.uniform(0, 100_000), -3),
        )

@asynccontextmanager
async def dummy_data(db, *, random_state=None):
    rnd = random_state if random_state is not None else Random()
    employees = [

```

```

        Employee.from_random(name=name, random_state=rnd)
    for name in 'Alice Bob Charlie Dana Evan Frank Gina Harry'.split()
]
create_table = dedent('''
    create table if not exists employees (
        name text
        , salary real
    );
''').strip()
insert = dedent('''
    insert into employees (name, salary) values (:name, :salary)
''').strip()
await db.execute(create_table)
await db.executemany(insert, (x._asdict() for x in employees))
yield db

```

```

@asynccontextmanager
async def lifespan(app):
    global db
    async with connect(':memory:') as db:
        async with dummy_data(db, random_state=Random(0)) as db:
            yield

```

```
app = FastAPI(lifespan=lifespan)
```

```

@app.get('/test')
async def test():
    return {'success': True}

```

```

@app.get('/data')
async def data():
    result = [row async for row in await db.execute('select * from employees')]
    return {'result': result}

```

```

class Question(BaseModel):
    question : str

```

```

@app.post('/question')
async def question(question : Question):
    root_url = 'https://api.openai.com/v1'
    openai_api_key = environ['OPENAI_API_KEY']
    layout = ';\\n'.join([
        x async for x, in await db.execute('select sql from sqlite_schema')
    ])

    async with AsyncClient() as client:
        url = f'{root_url}/responses'
        headers = {

```

```

        'Authorization': f'Bearer {openai_api_key}',
        'Content-Type': 'application/json',
    }
    json = {
        'model': 'gpt-4o',
        'input': question.question,
        'instructions': dedent(f'''
            Answer the specified question by writing SQL.
            Assume SQLite.
            Assume the below SQL table layout.
            Answer with just the SQL code.
            Do not add Markdown fenced code block garbage.

            {layout}

            {
                f"Here is an example of a good response: {history["accepted"][-1]}"
                if history["accepted"] else ""
            }

            {
                f"Here is an example of a bad response: {history["rejected"][-1]}"
                if history["rejected"] else ""
            }
        ''').strip(),
    }
    resp = (await client.post(url, headers=headers, json=json)).json()
    query = resp['output'][0]['content'][0]['text']
    query = query.replace('```sql', '').replace('```', '')
    tokens = resp['usage']['total_tokens']
    answer = [row async for row in await db.execute(query)]
    return {
        'question': question,
        'answer': answer,
        'query': query,
    }

history = {
    'accepted': [],
    'rejected': [],
}

class Acceptance(BaseModel):
    question : str
    query : str

@app.post('/accepted')
async def question(acceptance : Acceptance):

```

```

    history['accepted'].append(acceptance)
    return {'success': True}

```

```

class Rejected(BaseModel):
    question : str
    query : str

```

```

@app.post('/rejected')
async def rejected(rejected : Rejected):
    history['rejected'].append(rejected)
    return {'success': True}

```

```

if __name__ == '__main__':
    run(app)

```

```

{
  "scripts": {
    "serve": "vite serve --host 0.0.0.0 --port 8080 --debug --logLevel info --config vite.",
  },
  "dependencies": {
    "@b9g/crank": "^0.6.0",
    "extensible-runtime": "^4.1.0",
    "hyphen": "^1.10.6",
    "immutable": "^5.1.1",
    "uuid": "^11.1.0"
  },
  "devDependencies": {
    "@babel/plugin-proposal-decorators": "^7.25.9",
    "@babel/plugin-proposal-do-expressions": "^7.25.9",
    "@babel/plugin-proposal-function-bind": "^7.25.9",
    "@babel/plugin-syntax-jsx": "^7.25.9",
    "@babel/preset-env": "^7.26.9",
    "babel-plugin-extensible-destructuring": "^4.3.1",
    "vite-plugin-babel": "^1.3.0",
    "vite-plugin-ignore-public": "^0.0.3",
    "vite-require": "^0.2.3"
  }
}

```

```

import { defineConfig } from 'vite'
import babel from 'vite-plugin-babel'
import { viteRequire } from 'vite-require'
import { IgnorePublicPlugin } from 'vite-plugin-ignore-public'

export default defineConfig({

```

```

publicDir: 'public',
server: {
  allowedHosts: true,
  proxy: {
    '/api': {
      target: 'http://localhost:8000',
      changeOrigin: true,
      rewrite: (path) => path.replace(/^\/api/, ''),
    }
  }
},
esbuild: {
  jsx: 'transform',
  jsxDev: true,
  jsxImportSource: '@',
  jsxInject: "import { createElement, Fragment } from '@b9g/crank'",
  jsxFactory: 'createElement',
  jsxFragment: 'Fragment',
},
plugins: [
  IgnorePublicPlugin(),
  viteRequire(),
  babel({
    babelConfig: {
      sourceMaps: true,
      presets: [
        ['@babel/preset-env', {modules: false}]
      ],
      plugins: [
        '@babel/plugin-syntax-jsx',
        ['@babel/plugin-proposal-decorators', {version: '2023-11'}],
        '@babel/plugin-proposal-do-expressions',
        '@babel/plugin-proposal-function-bind',
        ['babel-plugin-extensible-destructuring', {mode: 'optout', impl: 'immutability'}]
      ]
    }
  )
],
})
})

```

```

<html>
  <body>
    <script src="index.jsx" async type="module"></script>
  </body>
</html>

```

```

import { renderer } from '@b9g/crank/dom'
import axios from 'axios'

async function *App() {
  let inputRef = undefined
  let response = {
    query: undefined,
    answer: undefined,
  }
  const onClick = {
    submit: async () => {
      const { value: question } = inputRef
      const { data : { query, answer } } = await axios.post('/api/question', { quest
response.query = query
response.answer = answer
      this.refresh()
    },
    okay: async () => {
      const { value: question } = inputRef
      await axios.post('/api/accepted', { question, query: response.query })
    },
    notOkay: async () => {
      const { value: question } = inputRef
      await axios.post('/api/rejected', { question, query: response.query })
    },
  }
  for ({ } of this) {
    console.log({ response })
    yield <>
      <h1>App</h1>
      <h2>Question</h2>
      <input type="text" ref={x => inputRef = x } />
      <button onClick={ onClick.submit }>
        Submit
      </button>
      <h3>Query</h3>
      <p>{ response.query }</p>
      <h3>Answer</h3>
      <p>{ response.answer }</p>
      <button onClick={ onClick.okay }>Okay</button>
      <button onClick={ onClick.notOkay }>Not Okay</button>
    </>
  }
}

await renderer.render(
  <App />,

```

```
    document.body,  
)
```

Building Systems Within LLMs

Key Questions

- How do we build systems that allow for human and machine supervision of LLM tasks?

```
print("Let's take a look !")
```

```
answer = []  
answers.append(ask_question(question))  
answers.append(ask_question('Please explain the answer to the question {question} {answer[0]}'))  
answers.append(ask_question('Where might this answer {answer[0]} give us a misleading answer?'))
```

```
ask_question(  
    question,  
    next_step={  
        lambda ans: ...: ask_question(...),  
        lambda ans: ...: ask_question(...),  
    }  
)
```

```
questions = {  
    'a': ('gpt-4o', 'What ...?'),  
    'b': 'How ...?',  
    'c': 'Where ...?',  
}  
  
conversation = {  
    'a': {  
        lambda ans, tok, tm, mod: ...: 'b',  
        lambda ans, tok, tm, mod: ...: 'c',  
    },  
    'b': {  
        lambda ans, tok, tm, mod: ...: 'a',  
    },  
    'c': {  
        lambda ans, tok, tm, mod: ...: 'a',  
    },  
}
```

```

def execute(questions, conversation, start='a'):
    state = start
    while state:
        q = question[state]
        answer = ask_question(q)
        for pred, next_state in conversation.items():
            if pred(answer):
                state = state_next
                break
        else:
            state = None

```

```

def conversation():
    for _ in range(3):
        answer = yield 'What...?'
        if answer == ...:
            answer = yield 'How...?'
        else:
            answer = yield 'Where...?'

```

```

from asyncio import run, gather
from collections import namedtuple
from itertools import product
from os import environ
from textwrap import dedent

```

```

from httpx import AsyncClient
from numpy import linspace

```

```

class Request(namedtuple('Request', 'input instructions model temperature')):
    def __new__(cls, input, instructions=None, model='gpt-4o', temperature=1):
        return super().__new__(cls,
                                input=input,
                                instructions=instructions,
                                model=model,
                                temperature=temperature,
                                )

```

```

Response = namedtuple('Response', 'output tokens')

```

```

def conversation(question, layout):
    models = {'gpt-3.5-turbo', 'gpt-4o'} #, 'o1'}
    initial_responses = yield [
        Request(
            input=question,

```

```

        instructions=dedent(f'''
            Answer the specified question by writing SQL.
            Assume SQLite.
            Assume the below SQL table layout.
            Answer with just the SQL code.
            Do not add Markdown fenced code block garbage.

            Layout:
            {layout}
        ''').strip(),
        model=model,
        temperature=temperature,
    )
    for temperature, model in product(
        (x.item() for x in linspace(0, 1, 3)),
        models,
    )
]
print(*initial_responses, sep='\n')
return
yield [Request(
    input=dedent(f'''
        Please explain the following query {initial_response.output}
        which answers the question {question}
    ''').strip(),
    instructions=dedent(f'''
        Just give the explanation. Don't add any additional conversation or chattiness
        Be succinct.
    ''').strip(),
)]
yield [Request(
    input=f'''
        Please explain how the following question might
        give a misleading answers given the following query.

        Query:
        {initial_response.output}
        Question:
        {question}
    ''',
    instructions=dedent(f'''
        Just provide a short description of the possible errors or misleading answers.
        Provide a corrected query for each case.
        Provide the corrected query after each possible error (not at the end.)
    ''').strip(),
)]

```

```

async def main(

```

```

root_url='https://api.openai.com/v1',
openai_api_key=environ['OPENAI_API_KEY'],
):
    conv = conversation(
        question='Where is the biggest gap in salary (and for whom) considering only emplo
        layout=''
            create table if not exists employees (
                name text
                , salary real
            );
        ''',
    )

    headers = {
        'Authorization': f'Bearer {openai_api_key}',
        'Content-Type': 'application/json',
    }
    json = {}

    async with AsyncClient() as client:
        resps = None
        while True:
            try:
                queries = conv.send(resps)
                tasks = [
                    client.post(f'{root_url}/responses', headers=headers, json={
                        **json,
                        **q._asdict(),
                    })
                    for q in queries
                ]
                api_resps = [x.json() for x in await gather(*tasks)]
                resps = [
                    Response(
                        output=r['output'][0]['content'][0]['text'],
                        tokens=r['usage']['total_tokens'],
                    )
                    for r in api_resps
                ]
                # print(f'{query.input}')
                # print(f'{resp.output}')
            except StopIteration:
                break

if __name__ == '__main__':
    run(main())

```

```
from numpy import linspace
```

```
for x in linspace(0, 1, 5):  
    print(f'{x.item() = }')
```